COURSEWORK 2: AUDIO IDENTIFICATION

Jimena Arruti
Student ID - 25029109
j.arruti@se25.qmul.ac.uk

1. INTRODUCTION

Given a short audio fragment – the query – and a database of complete recordings, the audio identification task consists of retrieving the original recording the query is reproducing. This matching process is conducted by comparing specifically designed audio features called *fingerprints*.

An audio identification algorithm was implemented based on the one described by Wang et al. in [1] from the company Shazam [1], as well as the remarks made by Müller on [2], and adapted to return the best three matches from the relevant database.

The system is designed to operate on a subset of the GTZAN dataset, comprising 100 pop recordings and 100 classical recordings. 108 degraded snippets from the classical recordings and 105 degraded snippets from the pop recordings were used as query examples. The snippets aim to simulate the real-world conditions in which the system would be used, containing distorted or filtered audio at a low volume, combined with everyday sounds.

2. RELATED WORK

Extensive research has been conducted to address audio identification by designing fingerprints, including classical signal processing approaches as well as more recent work involving deep learning. For this work, we will focus on the first group.

Work on audio identification and fingerprinting has included a broad range of feature selections, as well as hashing and matching techniques. For instance, Cano et al. developed a fingerprinting method called AudioDNA, designed to be robust to several types of distortions found during radio broadcasting [3]. They relied on extracting Mel-Frequency Cepstral Coefficients (MFCCs) and constructed an acoustic model based on Hidden Markov Models (HMM). In contrast, Haitsma and Kalker proposed a system envisioned to be able to identify a song by recording a fragment with a mobile phone [4]. The analysed feature in this case is spectral power density, processed using a Bark scale filter bank. Dupraz and Richard later focused on extending existing techniques to support searches when speed changes occurred and reducing fingerprint complexity to speed up the matching process [5]. In addition, Ramona and Peeters propose the fingerprint computation based on two passes of STFT calculations, where frequency bands are grouped in Bark bands, obtaining a vector which is then processed by a Nearest Neighbor Algorithm [6]. Finally, Joren and Leman designed Panako, a system that additionally contemplates changes in speed and pitch [7].

The present paper will focus on an implementation of the system described by Wang and used by Shazam [1], since there will not be changes in speed or pitch on the query recordings.

[1] https://www.shazam.com/

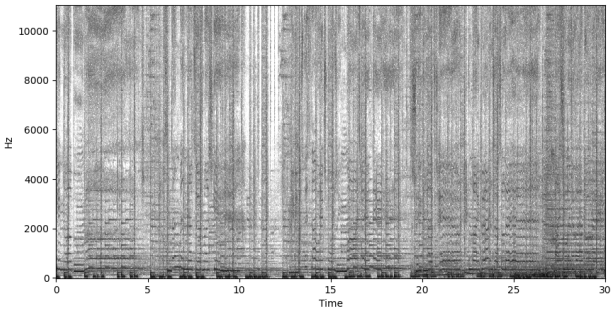


Figure 1. STFT of $pop.00000.wav$, with $window_length = 1024$, $hop_size = 512$ and $n_fft = 1024$.

3. IMPLEMENTATION

The implemented system can be divided in three parts: 1) Processing an audio signal to obtain spectro-temporal peaks that we will call *constellations*. 2) Creating hashes by combining the points of each obtained *constellation*, in order to store and access the information efficiently. 3) Running a matching algorithm that, starting from the hashes of a degraded short recording (the query), compares it to the hashes of all possible recordings in the database and returns the best three matches.

3.1 Audio processing

Every recording is processed by calculating its Short-Time Fourier Transform (STFT). The audio is divided into windows of 2048 samples with a hop size of 25%, and the FFT is calculated with 4096 frequency bins. An example of STFT for a pop recording can be found in Figure 1.

After the STFT is calculated, a peak picking algorithm selects the time-frequency coordinates where local maxima occur. *sci-kit image*'s $peak_local_max$ function is used to find the local maxima, having the possibility of selecting the minimum distance between peaks and a relative threshold for the minimum amplitude a peak should

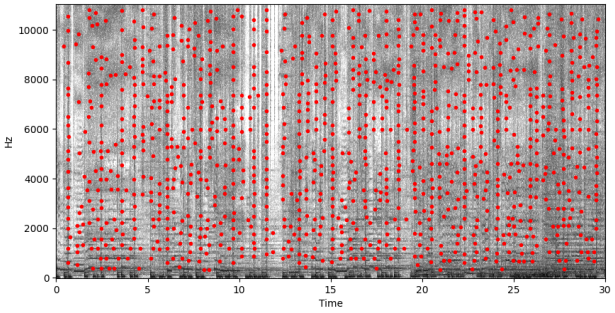


Figure 2. Constellation found for the STFT of Figure 1. Result obtained using $min_distance = 10$ and $threshold_rel = 0.05$

have. The resulting group of coordinates of local maxima is referred to as a *constellation*. The overlay of the detected *constellation* and the original STFT for the same pop recording can be seen on Figure 2,

3.2 Hashing

To simplify the implementation, given that the database is small, the function *fingerprintBuilder* constructs a dictionary containing the hashes for all the recordings in the database and saves it to a JSON file.

In order to have a fast matching process while maintaining robustness and specificity, hashes are created by combining the information of pairs of points, as suggested in [1]. Each hash is formed by combining an anchor point (n_0, k_0) and a target point (n_1, k_1), obtaining the triple $(k_0, k_1, n_1 - n_0)$. This hash is treated as a key in the dictionary, with the name of the recording and the frame of the anchor point (n_0) as the stored values. If the same hash is obtained later, the values will be appended to a list for the same key.

Each point of the signal's constellation is iteratively considered an anchor. Assuming the points of the constellation are ordered in time, the relevant target points are the points of the constellation that are in the future and inside a *target zone* relative to the anchor point. In the present implementation, the *target zone* is imagined as a rectangle defined by selecting a relative start time index, the size in the time dimension and the size in the frequency dimension. Additionally, a *fan-out* factor is defined to limit the number of target points to be considered for each anchor point.

3.3 Matching

In order to match a query with the target recording in the database, the function *audioIdentification* first processes the query to obtain its corresponding hashes, following the procedure in 3.1 and 3.2.

For every recording in the database, the algorithm checks for coincidences in hashes, computing in that case the time difference between the time index of the query's hash (n_{0_q}) and the time index of the database recording's hash (n_{0_d}). This is based on the assumption that if the query is a degraded snippet of the database recording, the histogram of time differences $(n_{diff} = n_{0_q} - n_{0_d})$ should have a peak at the time difference corresponding to the offset between recordings. The presence of a peak in the histogram guarantees there is indeed a match and that it was not just a chance coincidence between hashes.

The counts for the time difference index that conforms the peak of the histogram are stored. After all the database has been scanned, the best three matches are the recordings for which the histogram peak's counts are the highest.

4. EVALUATION

4.1 Metrics

Considering the system returns the best three matches, ranked-based metrics are the most accurate way of evaluating the system.

The relevance function $\chi_{\mathcal{Q}}(r)$ is defined as,

$$\chi_{\mathcal{Q}}(r) := \begin{cases} 1, & \text{if } \rho_{\mathcal{Q}}(r) \in \mathcal{I}_{\mathcal{Q}}, \\ 0, & \text{if } \rho_{\mathcal{Q}}(r) \notin \mathcal{I}_{\mathcal{Q}}. \end{cases} \tag{1}$$

where $\mathcal{I}_{\mathcal{Q}}$ is the set of relevant documents to the query Q, and $\rho_{\mathcal{Q}}(r)$ is a list of ranked documents at rank r.

Using the definition in equation 1, we define precision and recall at rank r as

$$P_{\mathcal{Q}}(r) := \frac{1}{r} \sum_{k=1}^{r} \chi_{\mathcal{Q}}(k) \tag{2}$$

$$R_{\mathcal{Q}}(r) := \frac{1}{|\mathcal{I}_{\mathcal{Q}}|} \sum_{k=1}^{r} \chi_{\mathcal{Q}}(k) \tag{3}$$

Note that in this case there is only one correct document to retrieve, so recall can be expressed as

$$R^*_{\mathcal{Q}}(r) = \sum_{k=1}^{r} \chi_{\mathcal{Q}}(k) \tag{4}$$

Additionally, it's useful to remember the average precision for a given query, contemplating all ranks is

$$\overline{P}_{\mathcal{Q}} := \frac{1}{|\mathcal{I}_{\mathcal{Q}}|} \sum_{r=1}^{K} P_{\mathcal{Q}}(r) \chi_{\mathcal{Q}}(r) \tag{5}$$

which is also simplified in this case to

$$\overline{P}^*_{\mathcal{Q}} = \sum_{r=1}^{K} P_{\mathcal{Q}}(r) \chi_{\mathcal{Q}}(r) \tag{6}$$

We can then define the mean Average Precision (mAP) across all j queries as

$$\overline{P} := \frac{1}{J} \sum_{j=1}^{J} \overline{P}_{\mathcal{Q}_j} \tag{7}$$

We will analyse the values of recall at ranks 1, 2 and 3 to understand at which point the system retrieves the expected recording. However, we will aim to maximise the mean average precision (mAP) of our system, since it rewards higher-ranked correct predictions and it will be convenient to optimise the performance for a single value.

4.2 Optimisation

With the intention of maximising mAP, different combinations of the STFT parameters $window_length$, n_fft and hop_length were tested for an initial set of parameters of the hashing algorithm. The obtained average metrics can be seen in Table 1, together with the time it took to complete the matching of all the query examples. By looking at the table, we can say that if we don't have a time restriction, it would be a good decision to use $win_length = 2048$, $n_fft = 4096$ and $hop_length = 512$ since it's the combination that yielded the highest mAP. However, little performance is gained over the similar combination of parameters but with $n_fft = 2048$, for which the processing time is roughly half. In a scenario where the processing time should be minimised, this second combination would be preferred. Regarding time, it's worth noting the combination of $win_length = 1024$, $n_fft = 1024$ and $hop_length = 1024$ is the fastest one to process, but to a clear detriment in performance.

Table 1. Audio Identification Performance Metrics varying STFT parameters. The reported time corresponds to the processing time of identifying all the query set. Hashing parameters are $target_zone_time_start = 0$, $target_zone_size_time = 100$, $target_zone_size_frequency = 700$, $fan_out = 20$

window	n_fft	hop	R@1	R@2	R@3	mAP	Time (s)
1024	1024	256	0.6620	0.7136	0.7230	0.6909	24.59
		512	0.6338	0.6901	0.6995	0.6651	14.61
		1024	0.5258	0.5962	0.6056	0.5642	**8.49**
	2048	256	0.6620	0.7136	0.7230	0.6909	52.18
		512	0.6479	0.6901	0.7042	0.6737	24.69
		1024	0.5493	0.6056	0.6244	0.5837	12.05
	4096	256	0.6338	0.7089	0.7183	0.6745	75.75
		512	0.5962	0.6432	0.6526	0.6228	41.39
		1024	0.4695	0.5540	0.5822	0.5211	20.77
2048	2048	256	0.7324	0.7746	0.7840	0.7567	44.61
		512	0.7371	**0.7981**	**0.8028**	0.7692	27.90
		1024	0.6761	0.7418	0.7512	0.7121	13.18
	4096	256	0.7324	0.7793	0.7887	0.7590	87.69
		512	**0.7418**	**0.7981**	**0.8028**	**0.7715**	57.90
		1024	0.6714	0.7277	0.7371	0.7027	33.19

Having selected the best set of parameters for the STFT, another grid search for parameter combinations was conducted, this time varying the hashing parameters, and its results are shown in Table 2. The highest value of mAP is achieved by the parameters $target_zone_time_start = 0$, $target_zone_size_time = 200$, $target_zone_size_frequency = 200$ and $fan_out = 30$, and therefore those are the values that will be selected for the system.

A quick test was also conducted to explore the possibilities of using Mel-spectrogram instead of STFT, maintaining the parameters optimised for the STFT case and varying the number of Mel bands. The results can be seen in 3. Even though the performance of the system is lower than the one obtained with STFT, further testing with different combinations of parameters should be explored, as the time-frequency representation is now different. In particular, the hashing parameters should be further adjusted to reflect the differences in the new frequency scale.

After selecting the best model, an evaluation of the performance for the two different song styles was carried out. As shown in 4, the system works much better for pop songs than for classical music. This difference is likely explained when contrasting the characteristics of the two styles: pop music has more pronounced rhythmic patterns, prominence of vocals and compressed dynamic range, which could result in clearer and more stable constellation peaks when compared to classical music.

5. DISCUSSION

5.1 Design trade-offs

The system's design involves several trade-offs, including specificity, robustness, computational cost, and processing time. When analysing the results of Table 1, it is clear that up to a certain point, increasing the resolution of the STFT (with a larger window or a higher FFT size) improves the mAP value. This could be explained by the higher point density in constellation maps, which leads to more distinctive fingerprints and increases specificity. However, we must take into account that this approach may also introduce spurious peaks, which could hinder the system's robustness to certain degradations. In addition, in Table 2 it is evident that increasing the number of hashes by increasing the fan-out value will lead to more computation, and therefore the matching will take more time. On the other hand, increasing the fan-out and the size of the target zone also means more fingerprints could survive potential degradations, therefore increasing the robustness of the system. This shows that the cause of the increase in fingerprints is important to determine the implications.

It must be noted that the current implementation would potentially have problems with scalability, as it was designed for the scope of this task. A production-oriented system should operate using a database for storing the hashes instead of a JSON file, and would take the time restrictions into account when optimising the system.

5.2 Interpretation of results

The recall at different ranks metric provides additional insight on the system's behaviour. The small difference between recall at rank 1 and recall at rank 3 in 4 suggests that when the correct match is retrieved, it's often ranked at the top positions. The difference between recall at rank 1 and recall at rank 2 is higher than the difference between recall at rank 2 and recall at rank 3, which implies that there are few cases when the correct match is at rank 3. This shows that the errors are more likely due to a snippet not being matched rather than a ranking order imperfection.

As previously stated, all metrics are greatly higher for pop music. It is worth mentioning that only one snippet is not correctly matched for the case of pop music, while the situation is completely different for classical music. To further understand the limitations of the system, a qualitative analysis of failure cases was conducted by listening to

Table 2. Audio Identification Performance Metrics varying hashing parameters. The reported time corresponds to the processing time of identifying all the query set. STFT parameters are $window_length = 2048$, $hop_length = 512$ and $n_fft = 4096$

Start time	Time Size	Treq Size	Fan Out	R@1	R@2	R@3	mAP	Time (s)
0	50	200	10	0.7230	0.7606	0.7700	0.7449	35.75
			20	0.7559	**0.8122**	0.8122	0.7840	50.90
			30	**0.7606**	0.7981	0.8169	0.7856	69.53
		500	10	0.7230	0.7700	0.7746	0.7480	31.46
			20	0.7559	0.7934	0.8075	0.7793	56.92
			30	0.7512	0.7934	0.8028	0.7754	69.27
		700	10	0.7136	0.7606	0.7700	0.7402	32.11
			20	0.7418	0.7981	0.8028	0.7715	49.93
			30	**0.7606**	0.7934	0.8028	0.7801	68.07
	100	200	10	0.7230	0.7606	0.7700	0.7449	31.83
			20	**0.7606**	**0.8122**	0.8122	0.7864	50.48
			30	**0.7606**	0.8075	0.8169	0.7872	68.10
		500	10	0.7230	0.7700	0.7746	0.7480	28.62
			20	0.7512	0.7934	0.8075	0.7770	42.06
			30	0.7512	0.7934	0.8028	0.7754	58.71
		700	10	0.7136	0.7606	0.7700	0.7402	30.39
			20	0.7418	0.7981	0.8028	0.7715	48.79
			30	0.7559	0.7934	0.8028	0.7778	67.58
	200	200	10	0.7230	0.7606	0.7700	0.7449	35.47
			20	**0.7606**	**0.8122**	0.8122	0.7864	47.12
			30	**0.7606**	0.8075	**0.8216**	**0.7887**	69.45
		500	10	0.7230	0.7700	0.7746	0.7480	29.31
			20	0.7512	0.7934	0.8075	0.7770	42.74
			30	0.7512	0.7934	0.8028	0.7754	56.66
		700	10	0.7136	0.7606	0.7700	0.7402	**26.69**
			20	0.7418	0.7981	0.8028	0.7715	46.55
			30	0.7559	0.7934	0.8028	0.7778	66.55
50	200	200	30	0.7418	0.7934	0.8122	0.7739	70.27
100	200	200	30	0.7089	0.7465	0.7653	0.7340	78.13

Table 3. Performance using Mel Spectrogram, varying the number of Mel bands, using parameters optimised for STFT.

Mel bands	Recall @ 1	Recall @ 2	Recall @ 3	mAP	Time (s)
128	0.4695	0.5070	0.5305	0.4961	11.66
256	0.5775	0.6197	0.6338	0.6033	16.84
512	0.6620	0.6901	0.7089	0.6823	25.63
512 (Dense)*	**0.6714**	**0.7089**	**0.7089**	**0.6901**	50.01

* Using different peak pickings parameters: min_distance=5, threshold_rel=0.02

Table 4. Best model performance across styles

Style	Recall @ 1	Recall @ 2	Recall @ 3	mAP
All	0.7606	0.8075	0.8216	0.7887
Classical	0.6296	0.6389	0.6574	0.6404
Pop	**0.8952**	**0.9810**	**0.9905**	**0.9413**

the query recordings. In several instances involving classical music, the correct match was not retrieved within the top 3 results. In some of these cases the music is barely audible over the loud conversation level of the crowd during the whole recording, as is the case of snippets from classical-00009, classical-00019 and classical-00040. The system seems to be robust to other types of degradation, such as playback in different speakers (equivalent to filtering some frequency ranges), static noise, and eventual sounds like people coughing and car honks. As previously stated, when considering the nature of the music style, one could argue that the greater dynamic range and lower presence of percussive elements of classical music makes it prone to be easily masked. A good example of this reflected in a failure case is classical-00064, which is a piano piece played at a low dynamic.

5.3 Future work

Regarding future work, the current system could be additionally optimised by varying the parameters of the peak picking algorithm. More in-depth exploration could be conducted with Mel spectrograms, as the one performed was superficial and used fixed parameters optimised for the STFT. Furthermore, processing could be developed to work with MFCCs instead of only considering Mel spectrograms. Regarding new approaches, other spectro-temporal representations could be explored, such as the Constant Q Transform (CQT) or STFT with Bark bands.

6. CONCLUSION

An audio identification algorithm grounded on classical signal processing was successfully implemented, based on the work of Wang [1]. The system was designed to return the top-3 matches for every query, therefore recall at all three ranks is calculated, as well as mAP, for which the system is optimised. Parameters of the STFT computation and hashing creation were selected by a grid search method, with the resulting combination obtaining a mAP score of 0.7887 across all music styles. The performance

was shown to be better for pop music than for classical music, obtaining a mAP of 0.9413 and 0.6404 respectively.

7. REFERENCES

[1] A. Wang *et al.*, "An industrial strength audio search algorithm." in *Ismir*, vol. 2003. Washington, DC, 2003, pp. 7–13.

[2] M. Müller, *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Springer, 2015, vol. 5.

[3] P. Cano, E. Batlle, H. Mayer, and H. Neuschmied, "Robust sound modeling for song detection in broadcast audio," 04 2009.

[4] J. Haitsma and T. Kalker, "A highly robust audio fingerprinting system." in *Ismir*, vol. 2002, 2002, pp. 107–115.

[5] E. Dupraz and G. Richard, "Robust frequency-based audio fingerprinting," in *2010 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 2010, pp. 281–284.

[6] M. Ramona and G. Peeters, "Audio identification based on spectral modeling of bark-bands energy and synchronization through onset detection," in *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011, pp. 477–480.

[7] S. Joren and M. Leman, "Panako-a scalable acoustic fingerprinting system handling time-scale and pitch modification," in *Proc. of the Int. Society for Music Information Retrieval Conf.(ISMIR)*, 2014, pp. 259–264.